

Disciplina de Segurança em IoT

Prof. Charles Neu

Terceiro Trabalho

Data de entrega e apresentação: 30/06/2027

Objetivo: Aplicar os conceitos básicos de Segurança da informação em um cenário IoT, de modo a compreender como criar uma estratégia de proteção e comunicação segura, adicionando camadas de criptografia, *hash* e certificação digital.

Atividade: No Trabalho 1 foi criado um cenário IoT e mapeadas possíveis ameaças e ataques com uma breve discussão sobre formas de proteção. Foi adotado um modelo de ataque e mapeadas possíveis ameaças/ataques, os vetores e superfície de ataque e proposta uma estratégia de proteção. No Trabalho 2, foi adicionada uma arquitetura de proteção, utilizando algumas das principais ferramentas básicas: FIREWALL, IDS, PROXY, SIEM. Agora, no terceiro trabalho, devem ser adicionados os conceitos estudados em criptografia, trazendo mais segurança, autenticidade, confidencialidade e integridade aos dados. A tarefa consiste em implementar o cenário IoT proposto por cada grupo no início da disciplina e demonstrar a comunicação segura entre os dispositivos e o servidor.

Tarefa 1: consiste em desenvolver um simulador para seu cenário IoT

Desenvolver, em qualquer linguagem de programação (ex. Python, Java, etc) um software que simula a comunicação entre os dispositivos do seu cenário, com a geração e coleta de dados dos dispositivos (por exemplo, temperatura, e outros dados de sensores que existirem em seu cenário). Para isso, uma alternativa prática é a utilização de sockets para a comunicação dispositivo x servidor, mas podem usar qualquer alternativa que julgarem mais adequada. Não é necessário adicionar todos os dispositivos do seu cenário nesta tarefa, apenas 1 é o suficiente. Os demais serão adicionados na próxima etapa do trabalho. Vejam exemplo ao final deste documento.

Tarefa 2: Adicionar segurança à comunicação, com criptografia

Esta comunicação deve ser implementada de forma segura visando proteger a privacidade dos dados, usando os principais algoritmos de criptografia simétrica estudados aplicáveis em seu cenário. Cada grupo deve implementar o AES e pelo menos outros três métodos distintos adicionais, dentre os mais usados atualmente em IoT (faça uma pesquisa sobre quais são, como o chacha20) e fazer uma comparação entre eles (ex. Medir o overhead gerado em cada um, segurança, complexidade/simplicidade, possíveis tamanhos de chave e outras informações que julgarem importantes).

Tarefa 3: Implementação da função Hash

Escolha uma função hash adequada (por exemplo, SHA-256) e implemente-a em python (ou outra linguagem de programação qualquer).

Gere valores hash para pacotes de dados de amostra transmitidos por dispositivos IoT.

Verifique os valores de hash para garantir a integridade dos dados durante a transmissão, de modo que o servidor consiga verificar que os dados recebidos são íntegros e autênticos, que não foram violados, ou seja, de assegurar que as informações transmitidas não sofram alterações maliciosas ou ruídos no meio do caminho (*Tampering*). É necessário implementar uma função criptográfica de hash (Recomendado: **SHA-256**) e gerar o *digest* (hash) dos pacotes de dados antes do envio e validar o valor correspondente no recebimento pelo servidor.

Desafio extra: implemente autenticação(login/senha) segura em seu sistema, usando *hash*.

Tarefa 4: Certificados digitais

Explique o conceito de certificados digitais e sua aplicação na autenticação de dispositivos IoT.

Pesquise como gerar um par de chaves pública/privada (usando OpenSSL, por exemplo, ou outro qualquer), gere e demonstre como criar um certificado digital autoassinado para um dispositivo IoT usando as chaves geradas. Inclua informações como ID do dispositivo, chave pública, período de validade, etc. Simule a distribuição de certificados digitais para dispositivos IoT, de modo a prover uma identidade digital única para cada nó da rede IoT.

- descrever uma defesa teórica sobre o papel de certificados digitais na mitigação de ataques do tipo *Man-in-the-Middle* (MitM) em IoT.
- **Prática:** Utilizar ferramentas de criptografia (ex: **OpenSSL**) para gerar um par de chaves (Pública/Privada) e emitir um **Certificado Digital Autoassinado** para o dispositivo sensor.
- **Metadados Obrigatórios:** O certificado gerado deve conter explicitamente o *Device ID*, Chave Pública associada, Período de Validade e Autoridade Emissora Simulada.
- **Simulação:** Desenhar a lógica de distribuição desse certificado para o dispositivo final de maneira segura.

Tarefa 5: Geração e verificação de assinatura digital

Implemente a geração de assinatura digital usando criptografia assimétrica (por exemplo, RSA). Assine pacotes de dados de amostra com a chave privada correspondente ao certificado digital do dispositivo. Implemente a verificação de assinatura utilizando a chave pública extraída do certificado digital.

Tarefa 6: Protocolo de comunicação segura

Projete um protocolo de comunicação seguro para dispositivos IoT trocarem dados com o servidor central. Inclui mecanismos para criptografia de dados, verificações de integridade baseadas em hash e verificação de assinatura digital.

Implemente o protocolo usando MQTT, CoAP ou outro protocolo de comunicação compatível com IoT.

Tarefa 7: Teste e avaliação:

Utilize o ambiente simulado criado na tarefa 1 (o seu cenário IoT).

Transmita pacotes de dados de amostra usando seu protocolo de comunicação seguro.

Monitorar o processo de comunicação e analisar a eficácia dos mecanismos de segurança, avaliar a sobrecarga (overhead).

OBS: Os grupos que tiverem acesso a dispositivos como Raspberry Pi e sensores, podem utilizá-los neste trabalho para uma demonstração mais realista. Quem não tiver o hardware necessário disponível, pode apenas implementar os simuladores via software, ou solicitar ao professor para avaliar disponibilidade.

Tarefa 8: Relatório

Escrever um relatório técnico (padrão SBC) entre 6 e 8 páginas, descrevendo o trabalho desenvolvido. Sugere-se a seguinte estrutura:

- Resumo,
- Capítulo 1: Introdução (Contextualizar a necessidade de segurança em IoT, escrever os objetivos deste trabalho, o que será apresentado),
- Capítulo 2: Referencial teórico (descrever o que é IoT, o que é comunicação Máquina-Máquina – M2-M, explicar os algoritmos/métodos usados {ex. hash, chacha20, aes, RSA, etc}),
- Capítulo 3: Descrever (podem copiar do T1 e T2 a parte do cenário já feita nestes) o cenário e ilustrar as ferramentas de proteção e adicionar texto explicando o que foi implementado neste trabalho 3. Descrever como cada uma das tarefas deste trabalho 3 foram implementadas (podem adicionar trechos de código, telas, etc).

- Capítulo 4: Faça uma discussão sobre o uso destas camadas de segurança, discutindo vantagens, desvantagens, problemas/benefícios, necessidade, etc. da criptografia e outras tecnologias usadas. Discursar sobre os ganhos das camadas de segurança adicionadas e comparar com o cenário se estas camadas não estivessem implementadas (podem fazer uma comparação). Descrever como poderiam ocorrer alguns ataques, e criar uma estratégia de proteção contra, contra, por exemplo, ataques *man-in-the-middle*.
- Capítulo 5: Conclusão.
- Apêndice: Adicionar um manual/passos-a-passo pra executar os códigos implementados e o próprio fonte.

Tarefa 9: Apresentação

Apresentar seu projeto como um todo, iniciando pela explicação do cenário todo. Incluir um overview de tudo que foi feito nos trabalhos 1, 2 e 3, com foco no 3. Focar especialmente em uma discussão sobre os ganhos que as implementações das camadas de segurança trazem ao cenário, discutindo, por exemplo, algum tipo de ataque que pode ser evitado.

Os trabalhos devem ser apresentados em aula para a turma toda. Cada grupo terá até 20 minutos para apresentar. Os trabalhos que não forem apresentados em aula não serão avaliados e terão nota ZERO.

Observe a seguir um exemplo de código, em PYTHON, para implementar um simulador de um cenário onde um servidor central recebe os dados de um dispositivo que possui sensores de temperatura e umidade. Nestes códigos, tem apenas uma estrutura básica para auxiliar vocês no entendimento da tarefa.

Exemplo de código básico para o servidor (criar estrutura de armazenamento):

```
import socket

# Configurações do servidor
HOST = '127.0.0.1' # Endereço IP do servidor
PORT = 65432      # Porta para escutar

# Cria o socket do servidor
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print("Servidor pronto para receber conexões...")
    conn, addr = s.accept()
    with conn:
        print('Conectado por', addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            print('Dados recebidos:', data.decode())
```

Exemplo de código básico para os sensores (um para temperatura e outro para umidade):

```
import socket
import time
import random

# Configurações do sensor
HOST = '127.0.0.1' # Endereço IP do servidor
PORT = 65432      # Porta do servidor

def send_data(data):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        s.sendall(data.encode())
        print(f"Dados enviados: {data}")

# Simulador de sensor de temperatura
def temperature_sensor():
    while True:
        temperature = random.uniform(20, 30)
        data = f"TEMPERATURE:{temperature}"
        send_data(data)
        time.sleep(2)

# Simulador de sensor de umidade
def humidity_sensor():
    while True:
        humidity = random.uniform(40, 60)
        data = f"HUMIDITY:{humidity}"
        send_data(data)
        time.sleep(2)

if __name__ == "__main__":
    # Iniciando threads para cada sensor
    import threading
    t1 = threading.Thread(target=temperature_sensor)
    t2 = threading.Thread(target=humidity_sensor)
    t1.start()
    t2.start()
```

Forma de Avaliação (os grupos que não entregarem o relatório ou não apresentarem o trabalho, receberão nota zero):

1. Tarefas 1 a 7: **SEIS** pontos (1 ponto cada, sendo que os alunos poderão computar 1 p.extra)
2. Relatório: **DOIS** pontos
3. Apresentação e demonstração do trabalho em aula: **1** ponto

OBS: Deve ser entregue o código fonte (pode ser link github ou similar), o relatório em forma de artigo no padrão SBC e os slides da apresentação. O trabalho poderá ser realizado em grupos (já definidos no início do semestre). Cada aluno deve entregar individualmente este trabalho na tarefa correspondente no moodle.